

Quick summary

- **NTRU** is a *lattice-based* public-key cryptosystem (post-quantum candidate).
 - In practice you use NTRU **to encrypt/encapsulate a symmetric key** (i.e., exchange a Master Key or a session key), not to replace symmetric keys entirely.
 - Typical pattern: **hybrid encryption / KEM** – use NTRU to securely transport a short symmetric key (MK), then use MK for fast AEAD (ChaCha20-Poly1305 / AES-GCM) for the bulk data.
 - NTRU can be used as PKE (encrypt arbitrary small message) or packaged as a **KEM** (encapsulate/decapsulate a randomly generated symmetric secret). KEM is recommended for key exchange use-cases.
-

1) Conceptual overview – what NTRU is

At a high level:

- NTRU works in a **ring of polynomials** (something like $R = \mathbb{Z}[x]/(\Phi(x))$, usually $\Phi(x)=x^N-1$ or similar) with small-coefficient polynomials as private material.
- The **private key** consists of small polynomials (kept secret).
- The **public key** is a polynomial computed from the private key and public modulus(s).
- **Encryption** uses a small random polynomial (blinding) and the public key to produce a ciphertext polynomial; **decryption** uses the private key and small-coefficient structure to recover the message (or the encapsulated secret).
- Security relies on the hardness of lattice problems (finding short vectors), thought to resist known quantum attacks better than RSA/ECC.

I avoid detailed algebraic formulas here – the important part for integration is the abstraction:

- `KeyGen()` $\rightarrow$ `(pk, sk)`
- `Encapsulate(pk)` $\rightarrow$ `(ciphertext, shared_secret)` // KEM
- `Decapsulate(sk, ciphertext)` $\rightarrow$ `shared_secret` // KEM

When used as a PKE:

- `Encrypt(pk, plaintext)` $\rightarrow$ ciphertext
 - `Decrypt(sk, ciphertext)` $\rightarrow$ plaintext
-

2) KEM vs PKE – which one to use?

- **PKE mode (classical):** encrypt an actual small message with the public key; receiver decrypts with private key. You could encrypt a symmetric MK with PKE.
 - **KEM (Key Encapsulation Mechanism):** the more modern canonical pattern for key exchange:
 - Sender calls `encapsulate(pk)` → receives two items: a ciphertext (to send) and a shared secret (a symmetric key).
 - Receiver calls `decapsulate(sk, ciphertext)` → recovers the same shared secret.
 - KEM is designed so you *don't* have to generate the symmetric key yourself and send it – the algorithm produces a shared secret and a matching ciphertext deterministically.
 - **Recommendation:** use **NTRU as a KEM** (or use a library variant that exposes KEM semantics). It is simpler, standardized for key exchange use-cases, and avoids mistakes with message formatting/padding.
-

3) Does NTRU generate the Master Key (MK), or does it exchange it?

- **NTRU KEM generates/encapsulates a symmetric secret** (effectively a generated key) and produces a ciphertext that the recipient can decapsulate to obtain the same secret.
 - In KEM mode you can think of the KEM as *generating* the MK (on sender-side during encapsulation) and delivering it securely (via ciphertext) to the recipient.
 - **NTRU PKE can be used to encrypt a user-generated MK**, but KEM is more common and safer for this purpose.
 - **So short answer:** NTRU (used as a KEM) *produces* a symmetric secret (MK) at encapsulation time and *transfers* it. You do not normally use NTRU to derive a long-term symmetric key deterministically – instead you use it to bootstrap symmetric keys.
-

4) Typical hybrid workflow (recommended for IoT)

This is the usual, secure pattern you should implement:

1. Provision / publish server public key

- Server generates NTRU keypair `(pk_server, sk_server)` (done securely on server). Server publishes `pk_server`.
- Device either knows/trusts `pk_server` via provisioning, or server signs it via a certificate chain.

2. Device wants to send encrypted payload

- Device runs `encapsulate(pk_server)` → gets `(ct, shared_secret)` :
 - `shared_secret` is a random symmetric key (e.g., 256 bits).
 - `ct` is the NTRU ciphertext to send to server.
- Device derives session keys (e.g., `symmetric_key = HKDF(shared_secret, info)`) and uses AEAD (ChaCha20-Poly1305 or AES-GCM) with that symmetric key to encrypt telemetry/payload.
- Device sends `{ct, AEAD(ciphertext, associated_data)}` to the server. Include a counter/nonce and device identifier in associated data.

3. Server receives

- Server uses `decapsulate(sk_server, ct)` → recovers `shared_secret`.
- Server derives the same `symmetric_key = HKDF(shared_secret, info)` and runs AEAD decryption on the payload.
- If AEAD tag verifies, payload integrity and authenticity (w.r.t symmetric key) guaranteed.

This pattern provides confidentiality (NTRU protects the symmetric key against quantum adversaries), integrity (AEAD), and efficiency (symmetric encryption used for bulk).

5) Where to run KeyGen (server or device)?

Pros & cons

Options:

1. Server keypair (recommended common pattern)

- Server generates `(pk, sk)` . Server holds `sk` (high security). Devices obtain `pk` (public).
Devices encapsulate/enc encrypt MK to server.
- Pros: low device CPU, simple provisioning (set pk once), server handles heavy crypto.
- Cons: if device needs to receive secrets from server, server must encapsulate using device's public key or use authenticated channel.

2. Device keypair

- Device has `(pk_device, sk_device)` . Server uses `pk_device` to encapsulate MK to device.
- Pros: stronger device-side secret ownership; server cannot impersonate device.
- Cons: keygen may be expensive on tiny devices; provisioning becomes more complex.

For most constrained IoT devices, prefer **server-side keypairs** (server publishes public key) and devices perform encapsulation. If device has secure element capable of key storage and generation, device keypairs are an option.

6) Practical implementation details & recommendations

Security and correctness require attention to many details:

A. Use KEM (recommended)

- Prefer the KEM API if your library exposes it (encapsulate/decapsulate). It's simpler and less error-prone than encrypting a raw symmetric key.

B. Combine with HKDF (don't use raw shared_secret directly)

- After encapsulation/decapsulation, feed `shared_secret` into an HKDF (with context info like purpose, device id, epoch) to derive actual AEAD keys and nonces:
 - `session_key = HKDF(shared_secret, "XenoCipher session", length)`
 - This defends against small biases and provides domain separation.

C. Use AEAD for payload

- Encrypt bulk data with ChaCha20-Poly1305 or AES-GCM using `session_key` . AEAD handles integrity and replay protection when you include counters/nonces as associated data.

D. Authentication & anti-MITM

- NTRU encryption provides confidentiality but not authentication. Prevent MITM:
 - Provision device to trust server public key (via secure provisioning).
 - Or use certificates / signatures or an authenticated channel for public key distribution.
 - Optionally include a device signature or HMAC (with device secret) for mutual authentication.

E. Nonce & replay protection

- AEAD needs unique nonces per key. Use a per-message monotonic counter (sent in header, MACed) or derive a nonce from HKDF(shared_secret || counter).

F. Key rotation & lifetime

- Use short-lived session keys. Re-encapsulate when counter exceeds a limit or after a time window. Keep MKs ephemeral if possible.

G. Side-channels & constant-time

- Use a vetted, constant-time implementation. Lattice crypto can leak side-channels if implemented naively; use established libraries and avoid your own low-level implementation.

H. Library choice

- Use well-maintained, audited libraries (OpenQuantumSafe/liboqs, well-known vendors). Don't implement NTRU from scratch.

I. Size & performance

- NTRU ciphertexts and public keys are larger than symmetric keys but much smaller than some lattice schemes – still, measure across your constrained network. Benchmark on device.

7) Example pseudocode flows

KEM-based (recommended)

Sender (Device):

```
(pk_server) // known
(ct, shared_secret) = NTRU_Encapsulate(pk_server)
session_key = HKDF(shared_secret, "XenoCipher session")
nonce = derive_nonce(session_key, counter) // unique per message
```

```
ciphertext = AEAD_Encrypt(session_key, nonce, plaintext, associated_data)
send { ct, counter, ciphertext, associated_data }
```

Receiver(Server):

```
shared_secret = NTRU_Decapsulate(sk_server, ct)
session_key = HKDF(shared_secret, "XenoCipher session")
nonce = derive_nonce(session_key, counter)
plaintext = AEAD_Decrypt(session_key, nonce, ciphertext, associated_data)
```

PKE-based (less recommended for KEX)

Sender:

```
mk = RAND(256-bit)
ct_mk = NTRU_Encrypt(pk_server, mk)
ciphertext = AEAD_Encrypt(mk, nonce, plaintext, aad)
send { ct_mk, nonce, ciphertext }
```

Receiver:

```
mk = NTRU_Decrypt(sk_server, ct_mk)
plaintext = AEAD_Decrypt(mk, nonce, ciphertext, aad)
```

KEM style is simpler and often more standard.

8) IoT constraints & performance tips

- **Device choice:** If the device is extremely resource constrained, consider:
 - using server-side keypairs (device only encapsulates)
 - offloading heavy ops to a secure element or co-processor if available
 - batching key exchanges (avoid frequent full public-key ops)
 - **Benchmark early:** try NTRU operations on the target hardware (keygen, encaps, decaps) and measure CPU/time/memory.
 - **Network payload sizing:** NTRU ciphertexts may increase packet size; pack carefully and consider fragmentation behavior of the radio.
 - **Fallback:** If device cannot afford NTRU, consider provisioning a short-lived symmetric credential/bootstrap via physical provisioning, then switch to symmetric-only updates.
-

9) Pros & cons of using NTRU in XenoCipher

Pros

- Post-quantum resistance for key exchange.
- Well-suited to hybrid schemes: cheap symmetric ops for bulk data, NTRU for bootstrap.
- Encapsulation makes key transport easy and less error-prone.

Cons / Challenges

- Implementation and side-channel concerns – use vetted libs.
 - Larger public keys/ciphertexts than symmetric-only schemes.
 - Device-side performance can be heavy on the weakest MCUs.
-

10) Final recommendations for your XenoCipher project

1. **Use NTRU as a KEM** to bootstrap MKs. Let the device call `encapsulate(pk_server)` and send the ciphertext to server. Server decapsulates and derives session keys.
 2. **Always run KEM output through HKDF** to derive AEAD keys and nonces.
 3. **Authenticate public keys** (trust on provisioning or via certificates) so there's no MITM.
 4. **Derive separate keys** for each subsystem (LFSR taps, transposition PRNG, AEAD) from MK using HKDF with distinct labels – this prevents cross-consumption bugs you saw earlier.
 5. **Benchmark on device** – confirm NTRU performance is acceptable; prefer server keypairs if device is very constrained.
 6. **Use well-maintained libraries** (liboqs/OpenQuantumSafe or other audited implementations) and compile for constant-time behavior where supported.
-

The short version (what we'll actually do)

1. **Keep only two cipher modes** to reduce complexity:
 - **Mode A (normal):** Your XenoCipher (Chaotic LFSR + Tinkerbell + Transposition), AEAD-wrapped with **ChaCha20-Poly1305** or **AES-GCM**.
 - **Mode B (attack):** **ChaCha20-Poly1305** only (or AES-GCM if you want hardware assist).
→ One strong, standardized fallback keeps the surface small.

2. Server-coordinated switching with a tiny control plane.

- Device does *lightweight* anomaly checks.
- Server does the heavy detection and sends a **signed SwitchCommand**.
- Device obeys only if the command passes signature/HMAC and local sanity checks.

3. Switch only at “epoch boundaries” with a grace window.

- Each packet header carries `{alg_id, epoch, counter}`.
- When switching, server announces **next_epoch**; both sides start Mode B at `epoch = next_epoch`.
- For N seconds or N packets, **dual-decrypt** is allowed to avoid race conditions.

4. Debounce everything.

- EWMA + consecutive breaches + cooldown → no thrashing.
- Minimum dwell time in each mode (e.g., ≥60 s).

These four keep false positives low, keep both ends in lock-step, and keep the cost bounded.

How to Overcome NTRU negatives ?

1) Kill false positives (cheaply) on ESP32

Use a 10 ms budget / check and < 20 KB RAM. Do this:

- **Metrics (constant-time, streaming):**

- **Entropy (8-bit):** rolling window of 512–1024 bytes; use a fixed-bin histogram and an approximate Shannon calc. Update incrementally.
- **Decrypt failures:** count failures per sliding 10 s window (ring buffer of timestamps).
- **Request rate:** token bucket (very cheap).
- **Timing:** keep an **EWMA** of encrypt/decrypt time (microsecond resolution) from `esp_timer_get_time()`.

- **Decision rule (debounced):**

- Maintain **EWMA** for each metric.
- A “breach” is `metric > baseline*α + β` (adaptive) or absolute thresholds if you insist.
- Require **K consecutive breaches** (e.g., K=3) in **T seconds** to set `ALERT=1`.
- Use a **cooldown timer**: after a switch, ignore alerts for 60 s.

2) State/compatibility: remove race conditions with epochs

Packet header (authenticated, 16 bytes AAD):

```
typedef struct __attribute__((packed)) {
    uint8_t  version;           // e.g. 1
    uint8_t  alg_id;            // 0 = Mode A (Xeno+AEAD), 1 = Mode B (ChaCha)
    uint16_t epoch;             // increments only on switch
    uint32_t counter;           // monotonically increasing per epoch
    uint32_t device_id;         // or short device handle
    uint16_t flags;             // bit0=grace, bit1=switch_pending, etc.
} xeno_hdr_t;
```

Rules that keep both ends in sync:

- **Switch happens only when epoch increments.** No mid-epoch algorithm change.
- **Server sends** `SwitchCommand{next_epoch, alg_id, not_before_ms}` signed (Ed25519) or HMAC'd with a long-term management key.
- Device:
 - Verifies signature/HMAC.
 - **Schedules** the switch at `next_epoch ≥ current_epoch + 1`.
 - Sets local `switch_pending` and pre-derives keys for `next_epoch`.

Grace window (dual-decrypt):

- For `GRACE_PACKETS` (e.g., 16) after the first packet with the new epoch arrives, the receiver:
 1. Tries AEAD verify with **new alg_id** using `(epoch, new_counter)`.
 2. If it fails and `hdr.flags & GRACE`, try **old alg_id** with `(epoch-1, old_counter)`.
- This absorbs in-flight packets and small reorderings.

Key derivation (deterministic, no round trips):

```
base = HKDF(MK, "xeno/base")
k_A(epoch) = HKDF(base, "A" || u16(epoch))
```

```
k_B(epoch) = HKDF(base, "B" || u16(epoch))
nonce = HKDF(base, "nonce" || u16(epoch) || u32(counter))[0..11]
```

- Both sides can precompute `k_A(epoch+1)` or `k_B(epoch+1)` once `SwitchCommand` is accepted.
 - **No desync** because epoch is explicit on the wire and authenticated.
-

3) Switching cost: make it predictable and tiny

- **One-time prep** per switch:
 - Pre-derive next keys (HKDF) — a few SHA256s, microseconds on ESP32.
 - **Pre-init cipher context** (mbedtls/ESP-IDF): allocate once, reuse.
- **Minimum dwell time** per mode: e.g., 60 s. No back-to-back switches.
- **Cap switch frequency**: at most 1 per 5 minutes (server-side policy).
- **Compute at message boundaries** only: switch at `counter == 0` of the next epoch.

On ESP32 this is very manageable: key derivation + context init << 1 ms typically, ChaCha20-Poly1305 runs ~100-200+ Mbps on ESP32-S3 in optimized builds; AES-GCM can use hardware AES on classic ESP32.

How it fits in ESP-IDF (FreeRTOS)

Core tasks

- **crypto_task**: encrypt/decrypt payloads. Reads current `(alg_id, epoch, counter)` from an atomic config.
- **telemetry_task**: pushes metrics into a small ring buffer and runs the **Detector** every 100 ms.
- **control_task**: receives `SwitchCommand` from server (MQTT/HTTPS), verifies, and updates the **Switch FSM**.

Tiny Switch FSM (device)

```
typedef enum { MODE_A, MODE_B } alg_e;
```

```

typedef struct {
    alg_e      active_alg;
    uint16_t   active_epoch;
    bool       switch_pending;
    alg_e      next_alg;
    uint16_t   next_epoch;
    uint64_t   not_before_us;
    uint64_t   last_switch_us;
    uint16_t   grace_packets;    // remaining
} switch_fsm_t;

```

Transition rules (pseudo):

```

on_detector_alert():
    // device-side local alert -> ask server (optional), or just raise fl
    publish_alert(metrics_snapshot);

on_switch_command(cmd):
    if verify(cmd.sig) && cmd.next_epoch > fsm.active_epoch
        && now_us >= cmd.not_before_us
        && now_us - fsm.last_switch_us > MIN_DWELL_US:
        fsm.switch_pending = true;
        fsm.next_alg      = cmd.alg_id;
        fsm.next_epoch    = cmd.next_epoch;
        prederive_keys(fsm.next_alg, fsm.next_epoch);
        set_flag_switch_pending_in_headers();

on_outgoing_message():
    hdr.version = 1;
    hdr.counter = ++counter;
    hdr.epoch   = fsm.active_epoch;
    hdr.alg_id  = fsm.active_alg;
    hdr.flags   = fsm.switch_pending ? FLAG_SWITCH_PENDING : 0;
    if (fsm.switch_pending && counter == 0) hdr.epoch = fsm.next_epoch;

on_receive_packet(hdr):
    if (hdr.epoch > fsm.active_epoch) {
        // switch boundary reached
        fsm.active_epoch   = hdr.epoch;
        fsm.active_alg     = server_announced_alg(); // from last accepted
        fsm.switch_pending = false;
        fsm.last_switch_us = now_us;
        fsm.grace_packets  = GRACE_PACKETS;
    }

```

```

}

if (!aead_decrypt(hdr.alg_id, hdr.epoch, hdr.counter)) {
    if ((hdr.flags & FLAG_GRACE) && fsm.grace_packets > 0) {
        // try previous epoch/alg
        if (try_prev_epoch_decrypt()) {
            fsm.grace_packets--;
        } else {
            record_decrypt_failure();
        }
    } else {
        record_decrypt_failure();
    }
}
}

```

This eliminates mode desync and thrashing.

ESP32 specifics (which algorithm when)

- **Fallback (Mode B):** prefer **ChaCha20-Poly1305** (fast on all ESP32 variants, no AES hardware dependency).
If you already rely on ESP32 AES hardware and your build uses mbedTLS with GCM acceleration, **AES-GCM** is fine.
 - **Normal (Mode A):** your XenoCipher wrapped in AEAD (use the same AEAD as Mode B to unify header/auth handling).
 - **Crypto libs:** use ESP-IDF mbedTLS for AEAD + HKDF; use your existing Tinkerbell/LFSR modules for Mode A internals (kept isolated).
-

How this answers the 3 risks

1) False positives

- Streaming EWMA + **consecutive breaches** + **server confirmation** + **cooldown & dwell**.
- Device alone never hard-switches without a verified server command (unless offline policy says otherwise).

2) State/compatibility complexity

- **Epoch-based switching** — change happens only on a clear boundary that's carried in the header and authenticated.
- **Grace dual-decrypt** — tolerates reordering and in-flight packets.
- **Deterministic KDF per (alg_id, epoch)** — keys never collide and need no handshake.

3) Switching cost

- Only two modes; **pre-derive keys**, pre-init contexts; **rate-limit** switches; switch at message boundaries.
 - On ESP32 the added cost is negligible compared to radio I/O.
-

Minimal configs to start with

- `GRACE_PACKETS = 16`
 - `MIN_DWELL = 60 s`
 - Detector cadence: **100 ms**
 - EWMA α : **0.2**
 - Switch only on **server-signed** command (Ed25519 or HMAC with management key).
 - Header AAD always authenticated by AEAD; include `{version, alg_id, epoch, counter, device_id, flags}`.
-

Optional: lightweight local-autonomous fallback (offline)

If device is **offline** and local detector shows sustained attack for, say, >120 s:

- Flip to Mode B **locally** but **do not increment epoch**; instead set `flags |= LOCAL_ALERT`.
- When server comes back, it will issue a proper SwitchCommand with a new epoch; device then aligns to the server.

This avoids bricking comms while still preventing obvious abuse.